TextureGuard AI: Klasifikasi Citra Tekstur DAGM 2007 dengan Fitur Local Binary Pattern (PyFeats) dan AutoML PyCaret

Laporan Ujian Tengah Semester — Pembelajaran Mesin

Nama	Evan Rafa Radya Alifian	**Kelas**	Machine Learning – E
NIM	240411100063	**Tanggal**	24 Oktober 2026

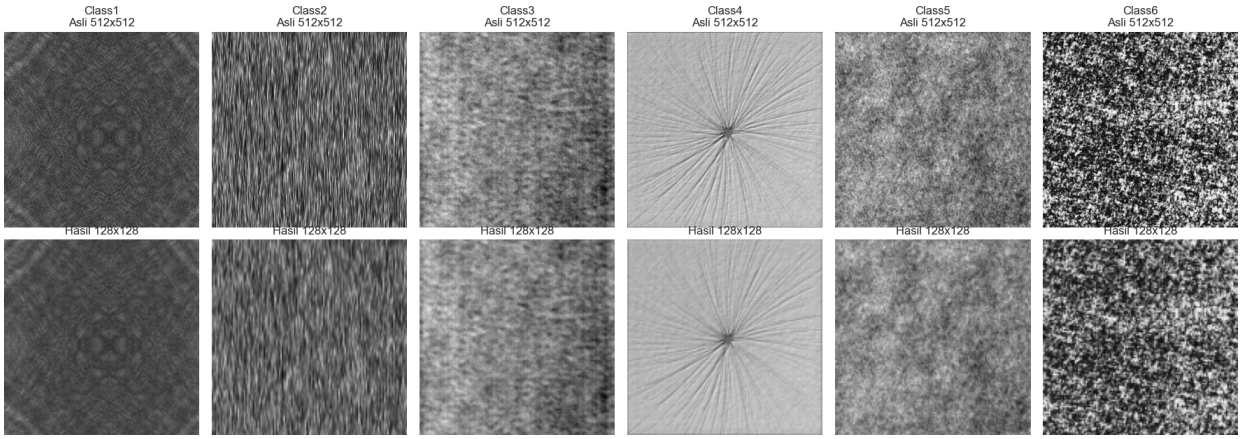
Ringkasan. Penelitian ini membangun *pipeline* klasifikasi citra tekstur berbasis *handcrafted features*. Citra dari kumpulan data DAGM 2007 (6 kelas, 6000 citra) dipra-proses, diekstraksi menjadi 12 fitur *Local Binary Pattern* (LBP) menggunakan PyFeats, lalu diklasifikasikan dengan AutoML PyCaret. Dari 13 model yang dibandingkan, *Quadratic Discriminant Analysis* (QDA) dan *Gradient Boosting Classifier* dipilih untuk di-*tuning*. Pada 1200 data uji, QDA mencapai akurasi **1,0000** dan Gradient Boosting **0,9917**. Fitur LBP skala kecil (R=1, P=8) dan skala menengah (R=4, P=16) paling berpengaruh terhadap prediksi.

1. Ringkasan Dataset

Dataset yang digunakan adalah **DAGM 2007** (*Weakly Supervised Learning for Industrial Optical Inspection*), yang disediakan oleh DAGM bersama Universitas Heidelberg dan diunduh dari https://conferences.mpi-inf.mpg.de/dagm/2007/prizes.html (berkas *Class1.zip* sampai *Class6.zip*). Setiap kelas memiliki latar tekstur buatan yang berbeda. Tugas dirumuskan sebagai klasifikasi **multikelas (6 kelas)**, dengan label diambil dari nama folder.

Tabel 1. Karakteristik dataset.

Karakteristik	Nilai
Jumlah kelas	6 (Class1 – Class6)
Jumlah citra	6000 (1000 per kelas, seimbang)
Format dan ukuran	PNG, grayscale 1 kanal, uint8, 512 × 512 piksel
Citra gagal dibaca / rusak	0
Pembagian data	Latih 4800 (800/kelas) dan uji 1200 (200/kelas), stratified 80 : 20



Gambar 1. Satu contoh tiap kelas: citra asli 512 × 512 (atas) dan hasil pra-pemrosesan 128 × 128 (bawah). Tiap kelas memiliki karakter tekstur yang berbeda.

2. Metodologi

2.1 Pra-pemrosesan citra

Setiap citra dibaca sebagai *grayscale*, diperkecil menjadi **128 × 128** piksel dengan interpolasi *INTER_AREA*, dan dipertahankan bertipe *uint8* (0–255) karena LBP membandingkan intensitas piksel dengan tetangganya. Augmentasi tidak digunakan karena data sudah besar dan seimbang, serta rotasi atau *flip* mengubah pola biner lokal yang menjadi dasar LBP. Normalisasi dilakukan pada level fitur di dalam pipeline PyCaret agar tidak terjadi kebocoran data.

2.2 Ekstraksi fitur: Local Binary Pattern (LBP)

LBP membandingkan intensitas setiap piksel pusat dengan *P* piksel tetangga pada lingkaran berjari-jari *R*. Tetangga yang intensitasnya ≥ pusat diberi bit 1 dan selainnya bit 0, sehingga terbentuk kode biner lokal (Ojala dkk., 2002). Fungsi *pyfeats.lbp_features* meringkas citra LBP tiap skala menjadi dua nilai, yaitu *energy* dan *entropy*. Agar pola pada berbagai skala tertangkap, digunakan **6 pasangan (P, R)** sehingga diperoleh **12 fitur** per citra.

Tabel 2. Konfigurasi LBP multi-resolusi.

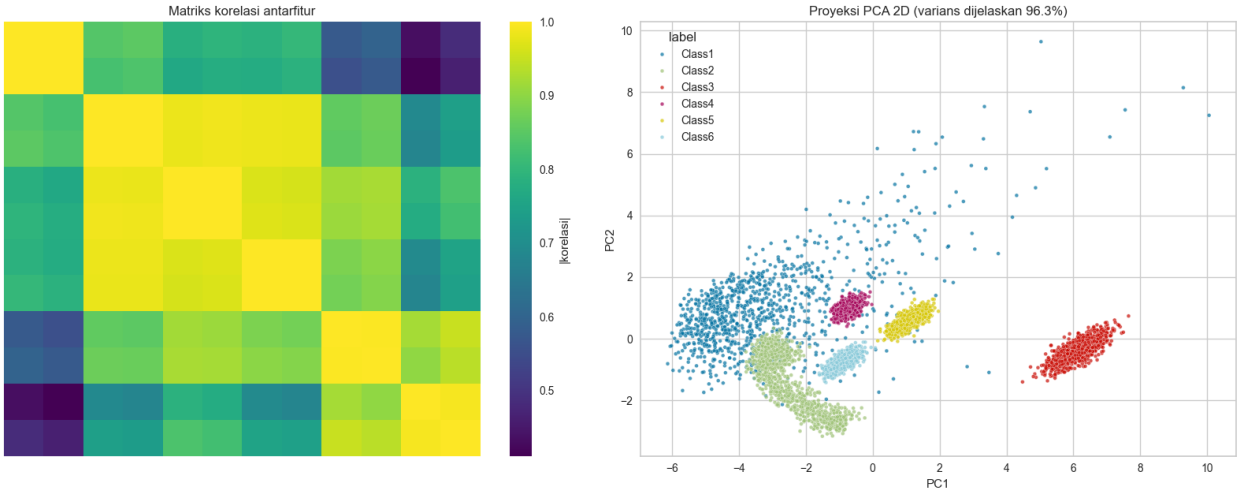
Skala	P (tetangga)	R (jari-jari)	Fitur
1	8	1	LBP_R_1_P_8_energy, _entropy
2	16	2	LBP_R_2_P_16_energy, _entropy
3	24	3	LBP_R_3_P_24_energy, _entropy
4	8	2	LBP_R_2_P_8_energy, _entropy
5	16	4	LBP_R_4_P_16_energy, _entropy
6	24	5	LBP_R_5_P_24_energy, _entropy

Catatan: PyFeats menghitung energy dan entropy langsung dari nilai kode LBP (bukan histogram ternormalisasi), sehingga entropy dapat bernilai negatif dan bukan entropi Shannon klasik.

2.3 Pembersihan dan partisi data

Ekstraksi fitur untuk 6000 citra berjalan paralel (*joblib*) dan selesai dalam waktu kurang dari satu menit, menghasilkan tabel *DataFrame* berukuran 6000 × 14 (label, nama berkas, 12 fitur). Hasil pemeriksaan kualitas data:

- **Missing value, nilai tak hingga, dan duplikat:** tidak ditemukan (0), sehingga tidak ada fitur atau baris yang dibuang.
- **Outlier (metode IQR):** 7.882 dari 72.000 nilai (10,95%) terdeteksi, terbanyak pada fitur *LBP_R_1_P_8*. Nilai ini tidak dihapus karena IQR dihitung pada gabungan semua kelas yang teksturnya sangat berbeda, sehingga nilai “ekstrem” kemungkinan besar merupakan ciri khas kelas lain, bukan kesalahan data. Pengaruhnya diredam dengan *robust scaling*.
- **Korelasi antarfitur:** 19 pasangan fitur memiliki |korelasi| > 0,95 (terutama *energy* dan *entropy* pada skala yang sama). PyCaret membuang fitur redundan, menyisakan **8 fitur**.
- **Partisi:** 80 : 20 secara *stratified* menjadi 4800 data latih dan 1200 data uji. Data uji tidak digunakan sama sekali pada *setup*, perbandingan model, maupun *tuning*.



Gambar 2. Matriks korelasi antarfitur (kiri) dan proyeksi PCA 2D (kanan; 96,3% varians). Class3 terpisah jauh, Class4–Class6 membentuk gugus rapat, sedangkan Class1 menyebar luas dan bersinggungan dengan Class2.

3. Pemodelan dengan PyCaret

3.1 Inisialisasi (*setup*)

Tabel 3. Pengaturan *setup()*.

Parameter	Nilai	Keterangan
data / target	4800 baris / label	Data latih saja
train_size, fold	0,8; 5-fold stratified	Hold-out internal 20% dan validasi silang
normalize	True (robust)	Skala berbasis median dan IQR, tahan outlier
numeric_imputation	median	Dipelajari hanya dari data latih
remove_multicollinearity	True (ambang 0,95)	12 fitur menjadi 8 fitur
session_id	42	Hasil dapat direproduksi

3.2 Perbandingan model dasar (*compare_models*)

Sebanyak 13 model dievaluasi dengan validasi silang 5-*fold* dan diurutkan berdasarkan *Accuracy*. *Dummy Classifier* (0,1667 = 1/6) menjadi garis dasar tebakan acak untuk 6 kelas; seluruh model lain berada jauh di atasnya.

Tabel 4. Hasil *compare_models()* (validasi silang 5-fold, data latih).

Model	Accuracy	Recall	Prec.	F1	Kappa	MCC
Quadratic Discriminant Analysis	0,9977	0,9977	0,9977	0,9977	0,9972	0,9972
Gradient Boosting Classifier	0,9891	0,9891	0,9892	0,9890	0,9869	0,9869
Extra Trees Classifier	0,9888	0,9888	0,9890	0,9887	0,9866	0,9866
Random Forest Classifier	0,9872	0,9872	0,9873	0,9872	0,9847	0,9847
K Neighbors Classifier	0,9865	0,9865	0,9871	0,9863	0,9838	0,9839
Naive Bayes	0,9818	0,9818	0,9830	0,9817	0,9781	0,9784
Decision Tree Classifier	0,9812	0,9812	0,9813	0,9812	0,9775	0,9775
Linear Discriminant Analysis	0,9773	0,9773	0,9777	0,9770	0,9728	0,9730
Logistic Regression	0,9682	0,9682	0,9680	0,9678	0,9619	0,9620
SVM – Linear Kernel	0,9578	0,9578	0,9605	0,9570	0,9494	0,9502
Ridge Classifier	0,9466	0,9466	0,9480	0,9456	0,9359	0,9366
Ada Boost Classifier	0,6174	0,6174	0,6523	0,5447	0,5409	0,5892
Dummy Classifier	0,1667	0,1667	0,0278	0,0476	0,0000	0,0000

Kolom AUC tidak dicantumkan karena bernilai 0,0000 pada beberapa model meskipun akurasinya tinggi, sehingga tidak dapat diandalkan; AUC yang valid dihitung secara manual pada data uji (Tabel 6). Dua baris tertinggi (disorot) dipilih untuk tuning.

3.3 *Hyperparameter tuning* (*tune_model*)

Dua model terbaik di-*tuning* dengan pencarian acak 20 iterasi (*optimize=Accuracy*, 5-*fold*) dan *choose_better=True*, yang hanya menerima hasil *tuning* bila lebih baik daripada model awal.

Tabel 5. Perbandingan akurasi validasi silang sebelum dan sesudah *tuning*.

Model	CV sebelum tuning	CV hasil tuning	Model yang dipakai
Quadratic Discriminant Analysis	0,9977	0,9875	Model awal
Gradient Boosting Classifier	0,9891	0,9904	Hasil tuning

Pada QDA, tuning tidak memperbaiki akurasi sehingga model awal dipertahankan. Pada Gradient Boosting, tuning menaikkan akurasi dari 0,9891 menjadi 0,9904 (simpangan baku antar-lipatan 0,0018).

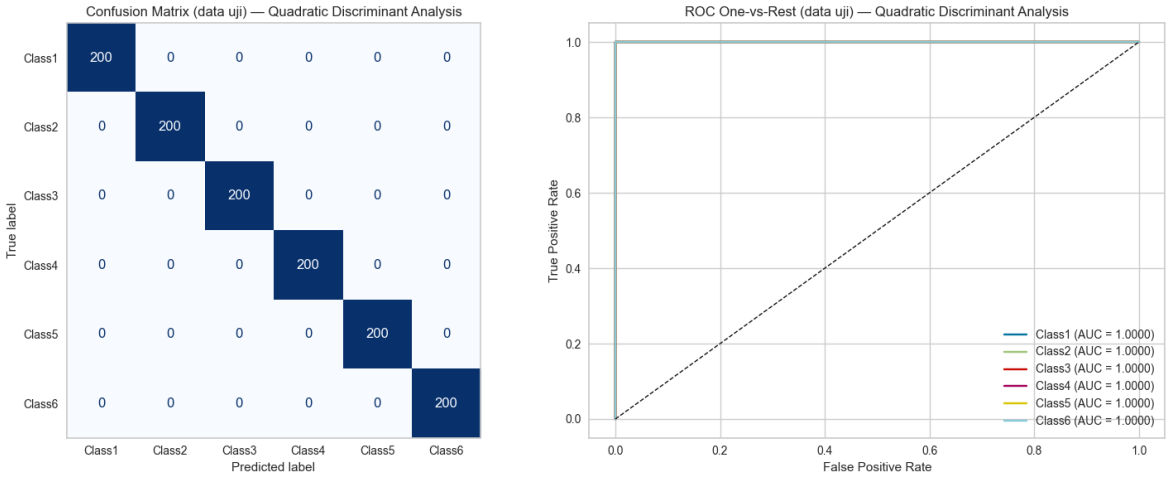
4. Hasil dan Evaluasi pada Data Uji

Kedua model hasil *tuning* diuji pada 1200 data uji yang belum pernah dilihat. AUC dihitung dengan skema *one-vs-rest* (rata-rata makro).

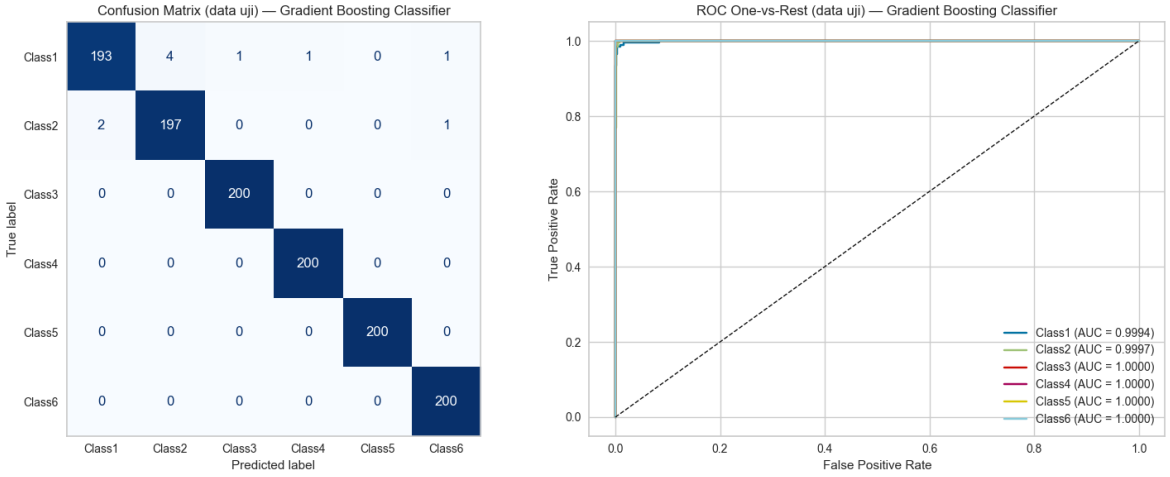
Tabel 6. Evaluasi akhir pada data uji (*predict_model*); Precision, Recall, F1, dan AUC adalah rata-rata makro.

Model	Accuracy	Precision	Recall	F1	AUC
Quadratic Discriminant Analysis	1,0000	1,0000	1,0000	1,0000	1,0000
Gradient Boosting Classifier	0,9917	0,9917	0,9917	0,9916	0,9998

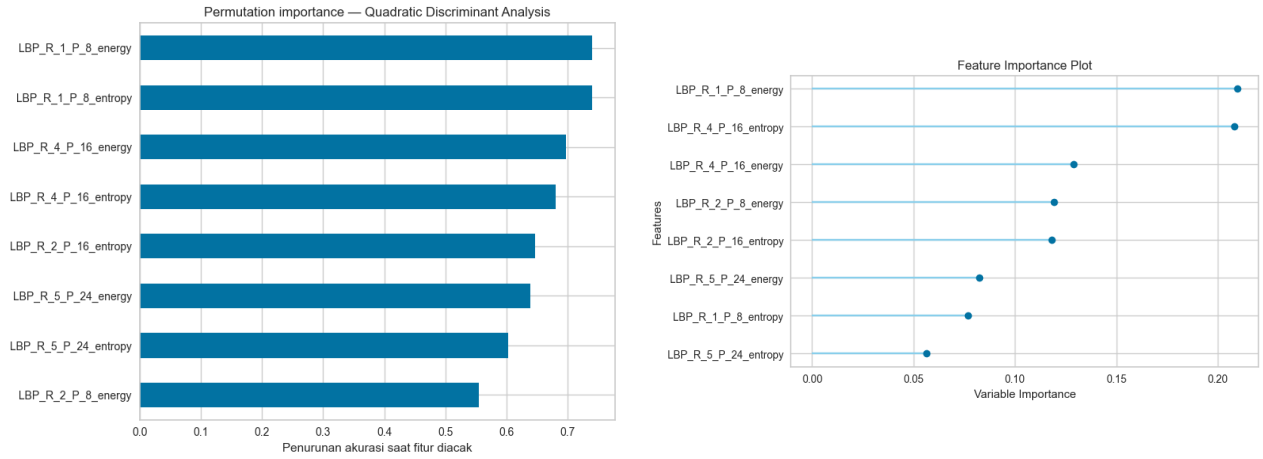
Model terbaik berdasarkan akurasi validasi silang adalah **QDA** (0,9977). Model ini dilatih ulang pada seluruh data latih (*finalize_model*), mencapai akurasi uji **1,0000**, dan disimpan sebagai *best_texture_model.pkl*.



Gambar 3. Confusion matrix dan kurva ROC one-vs-rest pada data uji: Quadratic Discriminant Analysis (tanpa kesalahan prediksi).



Gambar 4. Confusion matrix dan kurva ROC pada data uji: Gradient Boosting Classifier (10 kesalahan dari 1200; AUC Class1 = 0,9994, Class2 = 0,9997, kelas lain 1,0000).



Gambar 5. Feature importance: permutation importance pada QDA (kiri) dan variable importance pada Gradient Boosting (kanan), dihitung pada hold-out internal PyCaret.

5. Analisis Hasil

5.1 Fitur yang paling berpengaruh

Kedua model sepakat bahwa fitur LBP pada **skala kecil (R = 1, P = 8)** dan **skala menengah (R = 4, P = 16)** paling berpengaruh. Pada Gradient Boosting, *LBP_R_1_P_8_energy* dan *LBP_R_4_P_16_entropy* berada di puncak (sekitar 0,21), diikuti *LBP_R_4_P_16_energy* (sekitar 0,13). Pada QDA, pengacakan *LBP_R_1_P_8_energy* atau *_entropy* menurunkan akurasi paling besar (sekitar 0,74), disusul fitur skala R = 4. Fitur skala terbesar (R = 5, P = 24) cenderung paling kecil pengaruhnya.

Hal ini dapat dijelaskan oleh karakter citra pada Gambar 1. Pola yang paling membedakan kelas, misalnya guratan vertikal (Class2), bintik kontras tinggi (Class6), dan permukaan halus dengan garis memancar (Class4), tampak pada hubungan antar-piksel yang sangat berdekatan; ini ditangkap oleh LBP berjari-jari kecil. Skala menengah menambahkan informasi tentang keteraturan pola pada jarak yang lebih jauh. Selain itu, pada pengamatan ini *energy* dan *entropy* sama-sama muncul di jajaran atas, artinya keduanya memberi informasi yang saling melengkapi walaupun berkorelasi tinggi pada skala yang sama. *Catatan:* pada QDA seluruh fitur memperoleh nilai *permutation importance* yang tinggi (0,55–0,74) karena QDA memodelkan kovarians gabungan antarfitur, sehingga pengacakan satu fitur saja merusak model; peringkat antarfitur pada QDA perlu dibaca dengan hati-hati.

5.2 Akurasi model

Akurasi yang diperoleh sangat tinggi: QDA 100% dan Gradient Boosting 99,17% pada data uji, serta di atas 94% pada hampir semua model dasar. Hal ini konsisten dengan proyeksi PCA (Gambar 2) yang memperlihatkan kelas-kelas terpisah dengan baik. QDA unggul karena memodelkan setiap kelas sebagai sebaran Gaussian dengan kovarians sendiri, sesuai dengan gugus kelas yang rapat dan berbentuk elips pada ruang fitur. Kesalahan Gradient Boosting terkonsentrasi pada **Class1** (*recall* 0,965; 7 dari 200 salah) dan **Class2** (*recall* 0,985; 3 dari 200 salah), tepat pada dua kelas yang saling bersinggungan di PCA; Class3 sampai Class6 hampir sempurna. Pola ini menunjukkan bahwa kesalahan berasal dari tumpang-tindih sebaran fitur, bukan dari kelas yang sulit secara umum.

Nilai akurasi 100% perlu ditafsirkan hati-hati. Dengan nol kesalahan pada 1200 data uji, batas atas laju kesalahan pada tingkat kepercayaan 95% masih sekitar 0,25% (aturan tiga: 3/n), dan data uji berasal dari distribusi yang sama dengan data latih. Selain itu, tiap kelas DAGM memiliki latar tekstur yang sangat berbeda sehingga tugas klasifikasi antar-kelas relatif mudah.

6. Kesimpulan

- *Pipeline* lengkap (pra-pemrosesan, ekstraksi 12 fitur LBP dengan PyFeats, pembersihan dan partisi data, serta pemodelan PyCaret) berhasil mengklasifikasikan 6 kelas tekstur DAGM 2007 dengan akurasi tinggi.
- Dari 13 model, QDA dan Gradient Boosting menjadi dua model terbaik. *Tuning* meningkatkan Gradient Boosting (CV 0,9891 → 0,9904) namun tidak membantu QDA, sehingga model awal QDA dipertahankan.
- Model terbaik adalah **QDA** (CV 0,9977; akurasi, F1, dan AUC data uji 1,0000); Gradient Boosting mencapai akurasi uji 0,9917 dengan kesalahan terpusat pada Class1 dan Class2.
- Fitur paling berpengaruh adalah LBP skala kecil (R = 1, P = 8) dan menengah (R = 4, P = 16), sehingga kombinasi beberapa skala bermanfaat.
- **Keterbatasan dan saran:** kelas-kelas DAGM yang sangat berbeda membuat tugas relatif mudah dan skor mendekati sempurna. Penelitian lanjutan dapat mengklasifikasikan **ada atau tidaknya cacat** dalam satu kelas tekstur, menguji ketahanan terhadap derau dan pencahayaan, serta mempertimbangkan histogram LBP lengkap atau gabungan dengan metode lain karena ringkasan *energy*/*entropy* membuang informasi distribusi kode.

Sumber dan Referensi

DAGM 2007, *Weakly Supervised Learning for Industrial Optical Inspection*. https://conferences.mpi-inf.mpg.de/dagm/2007/prizes.html

Ojala, T., Pietikäinen, M., Mäenpää, T. (2002). Multiresolution gray-scale and rotation invariant texture classification with local binary patterns. *IEEE TPAMI*, 24(7), 971–987.

PyFeats: pustaka Python untuk ekstraksi fitur citra. https://pypi.org/project/pyfeats/

Ali, M. (2020). PyCaret: An open source, low-code machine learning library in Python. https://www.pycaret.org

Pedregosa, F. dkk. (2011). Scikit-learn: Machine learning in Python. *JMLR*, 12, 2825–2830.